

Assignment 7

Rollback and Commit commands:

Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

```
mysql> CREATE TABLE Teacher (  
->     Name VARCHAR(50),  
->     DeptNo INT,  
->     Date_of_Joining DATE,  
->     DeptName VARCHAR(50),  
->     Location VARCHAR(50),  
->     Salary INT  
-> );  
Query OK, 0 rows affected (0.12 sec)
```

(a) Insert five records

```
mysql> INSERT INTO Teacher VALUES  
-> ('Amit', 1, '2020-01-10', 'Mathematics', 'Delhi', 40000),  
-> ('Neha', 2, '2019-03-15', 'Commerce', 'Mumbai', 35000),  
-> ('Raj', 1, '2021-06-20', 'Mathematics', 'Delhi', 30000),  
-> ('Simran', 3, '2018-07-25', 'Science', 'Chennai', 45000),  
-> ('Karan', 2, '2022-02-10', 'Commerce', 'Mumbai', 28000);  
Query OK, 5 rows affected (0.02 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

(b) Give Increment of 25% salary for Mathematics Department.

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> SET AUTOCOMMIT=0;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> UPDATE Teacher  
-> SET Salary = Salary + (Salary * 0.25)  
-> WHERE DeptName = 'Mathematics';  
Query OK, 2 rows affected (0.01 sec)  
Rows matched: 2 Changed: 2 Warnings: 0
```

(c) Perform Rollback command

```
mysql> ROLLBACK;  
Query OK, 0 rows affected (0.01 sec)
```

(d) Give Increment of 15% salary for Commerce Department

```
mysql> UPDATE Teacher  
      -> SET Salary = Salary + (Salary * 0.15)  
      -> WHERE DeptName = 'Commerce';  
Query OK, 2 rows affected (0.00 sec)  
Rows matched: 2  Changed: 2  Warnings: 0
```

(e) Perform commit command

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.00 sec)
```

Assignment 8

(Exercise on order by and group by clauses):

Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

```
mysql> CREATE TABLE Sales (  
-> SalesNo INT,  
-> SalesName VARCHAR(50),  
-> Branch VARCHAR(50),  
-> SalesAmount INT,  
-> DOB DATE  
-> );  
Query OK, 0 rows affected (0.05 sec)
```

(a) Insert five records

```
mysql> INSERT INTO Sales VALUES  
-> (1, 'Amit', 'Delhi', 5000, '2001-12-21'),  
-> (2, 'Neha', 'Mumbai', 7000, '2000-05-10'),  
-> (3, 'Raj', 'Delhi', 6000, '1999-12-15'),  
-> (4, 'Simran', 'Chennai', 8000, '2002-08-25'),  
-> (5, 'Karan', 'Mumbai', 5500, '2001-12-05');  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

(b) Calculate total salesamount in each branch

```
mysql> SELECT Branch, SUM(SalesAmount) AS Total_Sales  
-> FROM Sales  
-> GROUP BY Branch;  
+-----+-----+  
| Branch | Total_Sales |  
+-----+-----+  
| Delhi  | 11000       |  
| Mumbai | 12500       |  
| Chennai | 8000        |  
+-----+-----+  
3 rows in set (0.01 sec)
```

(c) Calculate average salesamount in each branch

```
mysql> SELECT Branch, AVG(SalesAmount) AS Avg_Sales  
-> FROM Sales  
-> GROUP BY Branch;  
+-----+-----+  
| Branch | Avg_Sales |  
+-----+-----+  
| Delhi  | 5500.0000 |  
| Mumbai | 6250.0000 |  
| Chennai | 8000.0000 |  
+-----+-----+  
3 rows in set (0.00 sec)
```

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09

```
mysql> SELECT SalesName, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
-> FROM Sales
-> WHERE MONTH(DOB) = 12;
+-----+-----+
| SalesName | DOB      |
+-----+-----+
| Amit      | 21-Dec-01 |
| Raj       | 15-Dec-99 |
| Karan     | 05-Dec-01 |
+-----+-----+
3 rows in set (0.01 sec)
```

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
mysql> SELECT SalesName, DOB
-> FROM Sales
-> ORDER BY MONTHNAME(DOB);
+-----+-----+
| SalesName | DOB      |
+-----+-----+
| Simran    | 2002-08-25 |
| Amit      | 2001-12-21 |
| Raj       | 1999-12-15 |
| Karan     | 2001-12-05 |
| Neha      | 2000-05-10 |
+-----+-----+
5 rows in set (0.00 sec)
```

Assignment 9

Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES”. Their schemas are as follows :

Departments (dept_no , dept_name , dept_location); Employees (emp_id , emp_name , emp_salary,dept_no);

```
mysql> CREATE TABLE Departments (  
-> dept_no INT,  
-> dept_name VARCHAR(50),  
-> dept_location VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.15 sec)
```

```
mysql>  
mysql> CREATE TABLE Employees (  
-> emp_id INT,  
-> emp_name VARCHAR(50),  
-> emp_salary INT,  
-> dept_no INT  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

a) Develop a query to grant all privileges of employees table into departments table.

```
mysql> GRANT ALL PRIVILEGES ON Employees TO 'user1'@'localhost';  
Query OK, 0 rows affected (0.04 sec)
```

b) Develop a query to grant some privileges of employees table into departments table.

```
mysql> GRANT SELECT, INSERT ON Employees TO 'user1'@'localhost';  
Query OK, 0 rows affected (0.01 sec)
```

c) Develop a query to revoke all privileges of employees table from departments table.

```
mysql> REVOKE ALL PRIVILEGES ON Employees FROM 'user1'@'localhost';  
Query OK, 0 rows affected (0.01 sec)
```

d) Develop a query to revoke some privileges of employees table from departments table.

```
mysql> REVOKE INSERT ON Employees FROM 'user1'@'localhost';
```

e) Write a query to implement the save point.

```
mysql> SAVEPOINT sp1;  
Query OK, 0 rows affected (0.00 sec)
```

Assignment 10

Using the tables “DEPARTMENTS” and “EMPLOYEES” perform the following queries :

- a) Display the employee details, departments that the departments are same in both the emp and dept.

```
mysql> SELECT e.emp_name, e.emp_salary, d.dept_name
-> FROM Employees e
-> INNER JOIN Departments d
-> ON e.dept_no = d.dept_no;
Empty set (0.12 sec)
```

- b) Display the employee name and Department name by implementing a left outer join.

```
mysql> SELECT e.emp_name, d.dept_name
-> FROM Employees e
-> LEFT JOIN Departments d
-> ON e.dept_no = d.dept_no;
Empty set (0.00 sec)
```

- c) Display the employee name and Department name by implementing a right outer join.

```
mysql> SELECT e.emp_name, d.dept_name
-> FROM Employees e
-> RIGHT JOIN Departments d
-> ON e.dept_no = d.dept_no;
Empty set (0.00 sec)
```

- d) Display the details of those who draw the salary greater than the average salary

```
mysql> SELECT *
-> FROM Employees
-> WHERE emp_salary > (SELECT AVG(emp_salary) FROM Employees);
Empty set (0.01 sec)
```